Java Backend SWE1 – Oral Theory Answer Scripts

Rohit Jangir

1. Time and Space Complexity

Q: What is the time complexity of your solution?

Answer: My solution runs in $O(n)$ time because I scan the input once and perform only constant-time operations inside the loop. There are no nested iterations. The space complexity is $O(1)$ since I use only a few variables and do not allocate any extra data structures proportional to the input size.

2. HashMap Internal Working

Q: How does a HashMap work internally?

Answer: HashMap stores key–value pairs in an array of buckets. A hash function computes the bucket index. If collisions occur, the entries form a linked list, and from Java 8 onward, large chains are converted into a Red–Black Tree for better worst-case performance. Average-case time for lookup, insert, and delete is $O(1)$.

3. ArrayList vs LinkedList

Q: What is the difference between ArrayList and LinkedList?

Answer: ArrayList is backed by a dynamic array and provides $O(1)$ random access, but insertions or removals in the middle are slower. LinkedList is a doubly linked list, so insertions and deletions are $O(1)$ when the reference is known, but random access is $O(n)$.

4. Java 8 Features

Q: What are lambda expressions and functional interfaces?

Answer: A lambda expression provides a concise way to implement functional interfaces, which are interfaces containing exactly one abstract method. They enable cleaner code, especially for passing behavior, using streams, and simplifying callbacks.

Q: Difference between map() and flatMap()?

Answer: `map()` transforms each element into another value, returning a stream of the same structure. `flatMap()` is used when each element maps to another stream; it flattens all generated streams into a single output stream.

5. Concurrency

Q: What is the difference between Runnable and Callable?

Answer: Runnable does not return a result nor throw checked exceptions, while Callable returns a value and can throw checked exceptions. Callables are typically used with ExecutorService and Future.

Q: What does the volatile keyword do?

Answer: `volatile` guarantees visibility of changes across threads. A write to a volatile variable is immediately visible to other threads, preventing caching of stale values, but it does not provide atomicity.

Q: What is a race condition?

Answer: A race condition occurs when multiple threads access and modify shared data simultaneously, and the final result depends on the execution order. Proper synchronization or thread-safe structures prevent this.

6. Synchronized vs Lock

Q: Difference between synchronized and ReentrantLock?

Answer: `synchronized` is simpler, automatically releases the lock, and supports intrinsic locking. `ReentrantLock` offers more control, such as try-lock, timed lock, and fairness policies. It should be used when advanced locking behavior is needed.

7. OOP Concepts

Q: What is the difference between abstraction and encapsulation?

Answer: Abstraction hides complex implementation details and exposes only necessary functionality. Encapsulation binds data and methods together and restricts direct access using access modifiers. Abstraction focuses on *what* a system does; encapsulation focuses on *how* it is protected.

Q: Interface vs abstract class?

Answer: Interfaces specify contracts and allow multiple inheritance; methods are abstract by default. Abstract classes provide partial implementation and state and support single inheritance. Use interfaces for capabilities and abstract classes for shared behavior.

8. Design Patterns

Q: Explain the Singleton Pattern.

Answer: Singleton ensures only one instance of a class is created for the application's lifetime. It is implemented using a private constructor, a static instance, and a public getter. Often used for configuration, logging, and shared resources.

Q: Explain the Strategy Pattern.

Answer: Strategy encapsulates interchangeable algorithms behind a common interface. Clients can switch algorithms at runtime without modifying code. It promotes flexibility and follows the Open/Closed Principle.

9. Spring Boot

Q: What is Dependency Injection?

Answer: Dependency Injection is a design principle where Spring creates and manages object dependencies instead of manual instantiation. It improves modularity, testing, and decoupling by delegating object creation to the IoC container.

Q: @Component vs @Service vs @Repository?

Answer: All register beans in the Spring container. `@Component` is generic, `@Service` indicates business logic, and `@Repository` adds additional behaviors for data access such as exception translation.

10. Databases

Q: What is an index and why is it useful?

Answer: An index is a data structure (usually a B+ tree) that allows faster lookups by reducing search time from $O(n)$ to $O(\log n)$. However, it slows down writes because indexes must be maintained during inserts and updates.

Q: What is ACID in databases?

Answer: ACID stands for Atomicity, Consistency, Isolation, and Durability. They ensure reliable, predictable transactions even in failure scenarios.

11. DSA Concepts

Q: What is the difference between BFS and DFS?

Answer: BFS explores level by level using a queue and is good for shortest paths in unweighted graphs. DFS explores deep along one branch using a stack or recursion and is suited for cycle detection and topological sorting.

Q: What is dynamic programming?

Answer: Dynamic Programming solves problems by breaking them into overlapping subproblems and storing intermediate results. It avoids recomputation and typically uses either memoization or tabulation to achieve optimal time.

12. Observability (New Relic Focus)

Q: What is p95 latency?

Answer: p95 latency means 95% of requests are served faster than this value. It highlights tail latency and helps identify slow outliers impacting user experience.

Q: What are metrics, logs, and traces?

Answer: Metrics are numeric time-series data like latency or CPU usage. Logs are timestamped text records for events. Traces follow the path of a request across distributed services. Together they provide full observability of system behavior.